

TypeScript & Componentes en React

Guía práctica de clase · 90 minutos · Proyecto Vite + React + TS

Agenda de hoy

Bloque	Tema	Tiempo
■ 1	TypeScript: el porqué y el cómo	20 min
■ 2	tsconfig.json y el proceso de compilación	20 min
■ 3	¿Qué es un componente? Anatomía y ventajas	15 min
■ 4	Creando tus primeros componentes para tu proyecto	30 min
■ 5	Cierre, preguntas y reflexión	5 min

■ Objetivo de la clase

Al finalizar esta sesión podrás explicar qué hace TypeScript, leer un `tsconfig.json`, entender cómo Vite compila tu proyecto, y lo más importante: **crear, reutilizar y conectar componentes** en tu propio proyecto.

0 TypeScript: El porqué y el cómo

■ 20 min

1 Del JavaScript de siempre a un JavaScript con superpoderes

Antes de escribir una sola línea de código, necesitamos entender **qué problema resuelve TypeScript** y cómo lo hace. Esto cambiará la forma en que ves los errores en tu editor.

■ El problema con JavaScript puro

JavaScript es un lenguaje **dinámicamente tipado**: las variables no tienen un tipo fijo, y eso está bien para scripts pequeños. Pero en proyectos reales, ese mismo dinamismo causa bugs silenciosos que solo aparecen en tiempo de ejecución, muchas veces en producción.

JavaScript puro — error en runtime

```
// ■■ JavaScript — el error solo aparece al ejecutar
function saludar(nombre) {
  return "Hola, " + nombre.toUpperCase();
}

saludar(42); // ■ CRASH: nombre.toUpperCase is not a function
saludar(); // ■ CRASH: Cannot read properties of undefined
saludar("María"); // ■ Funciona bien
```

■ TypeScript al rescate

TypeScript agrega un sistema de tipos *sobre* JavaScript. El resultado: los errores se detectan **mientras escribes el código** en tu editor, no cuando el usuario ya vio el crash.

TypeScript — error detectado antes de ejecutar

```
// ■ TypeScript — el error aparece mientras escribes
function saludar(nombre: string) {
  return "Hola, " + nombre.toUpperCase();
}

saludar(42);

// ^^^^^ ■ Error en el editor: Argument of type 'number'
// is not assignable to parameter of type 'string'

saludar("María"); // ■ Perfecto, sin errores
```

■ Clave conceptual

TypeScript **no existe en el navegador**. Todo el código .ts y .tsx se convierte a JavaScript puro antes de ejecutarse. Lo que TypeScript hace es verificar tu código durante el desarrollo.

¿Cómo funciona el sistema de tipos?

TypeScript te permite declarar qué tipo de dato espera cada variable, parámetro o función. Hay dos formas de hacerlo:

Anotación explícita

```
let edad: number = 22
const nombre: string = 'Ana'
```

Tú dices el tipo explícitamente

Inferencia de tipos

```
let edad = 22
const nombre = 'Ana'
```

TS deduce el tipo solo

Ambos son válidos. La inferencia es más concisa, la anotación es más clara.

Tipos primitivos más usados

Tipo	Descripción	Ejemplo
<code>string</code>	Texto	<code>"Hola" 'mundo'</code>
<code>number</code>	Números enteros y decimales	<code>42 3.14 -7</code>
<code>boolean</code>	Verdadero o falso	<code>true false</code>
<code>string[]</code>	Array de strings	<code>["React", "Vue"]</code>
<code>number[]</code>	Array de números	<code>[1, 2, 3]</code>
<code>object</code>	Objeto genérico	<code>{ nombre: string }</code>
<code>void</code>	Sin valor de retorno	<code>function log(): void {}</code>
<code>any</code>	Cualquier tipo (evítalo)	<code>let x: any = 'todo vale'</code>

Interfaces y tipos personalizados

Una **interface** es como un molde que define la forma que debe tener un objeto. Es una de las características más poderosas de TypeScript para proyectos reales.

Interfaces TypeScript

```
// Definimos la forma de un producto (para el sitio de ventas)
interface Producto {
  id: number;
  nombre: string;
  precio: number;
  descripcion?: string; // el ? significa "opcional"
  disponible: boolean;
}

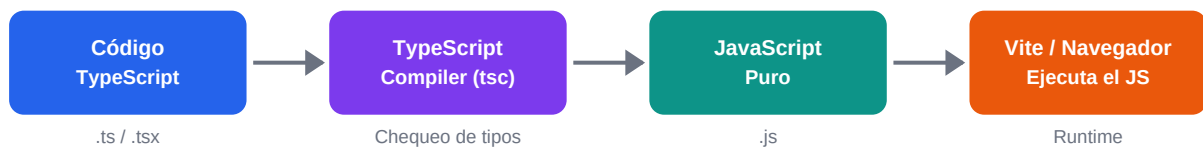
// Definimos la forma de un proyecto de portafolio
interface Proyecto {
  titulo: string;
  descripcion: string;
  tecnologias: string[];
  urlDemo?: string;
}
```

Regla de oro: si usas un objeto más de una vez (en diferentes componentes o funciones), créale una interface. Esto garantiza que siempre tenga la misma estructura y te ahorra bugs.


```
// ↑ Alias. @/components/Navbar en lugar de ../../components/Navbar
},
"include": ["src"]
// ↑ Solo compila los archivos dentro de src/
}
```

El flujo completo de compilación

Es fundamental entender lo que pasa desde que guardas un archivo .tsx hasta que el navegador lo ejecuta. Vite hace este proceso extremadamente rápido en desarrollo:



1 Escribes código TypeScript (.ts / .tsx)

Tu código tiene tipos, interfaces, anotaciones. Es TypeScript puro.

2 TypeScript Compiler verifica los tipos

El compilador (tsc) lee tu tsconfig.json y revisa que todos los tipos sean correctos. Si hay errores de tipo, te los reporta. No genera código si hay errores en modo strict.

3 Se genera JavaScript puro (.js)

TypeScript elimina todas las anotaciones de tipo. El resultado es JavaScript estándar. Las interfaces, los : string, los desaparecen completamente.

4 Vite sirve el JS al navegador

Vite toma el JS generado, lo optimiza, y lo sirve al navegador. En producción (npm run build) genera un bundle minificado en /dist.

■ ¿Por qué Vite es tan rápido?

En Vite, este proceso es casi instantáneo gracias a **esbuild** (escrito en Go). A diferencia de Webpack, Vite no transpila todo el proyecto al iniciar: solo transpila el archivo que solicitas. Por eso el servidor de desarrollo arranca en <1 segundo.

Diferencia entre .ts y .tsx

Aunque son casi iguales, existe una distinción importante:

Extensión	Cuándo usarla	Ejemplo
.ts	Lógica pura sin JSX: utils, tipos, hooks, servicios	formatDate.ts
.tsx	Componentes que retornan JSX (código con etiquetas HTML)	Navbar.tsx

.ts vs .tsx en la práctica

```
// utils/formatPrice.ts ← sin JSX, solo lógica
```

```
export function formatPrice(precio: number): string {
  return `$$${precio.toLocaleString("es-CO")}`;
}

// components/PriceTag.tsx ← tiene JSX, extensión .tsx
import { formatPrice } from "../utils/formatPrice";

export function PriceTag({ precio }: { precio: number }) {
  return <span className="price">{formatPrice(precio)}</span>;
}
```

0 ¿Qué es un Componente?

3

La unidad fundamental de React: reutilizable, independiente, composable

■ 15 min

Ya saben qué es JSX. Ahora vamos a entender **por qué los componentes son la idea más poderosa de React** y cómo TypeScript los hace aún más seguros y claros.

La analogía perfecta: piezas de LEGO



Cada componente es una pieza independiente

Así como una pieza de LEGO tiene una forma definida, un ensamble predecible y se puede usar en miles de construcciones distintas, un componente React tiene una **estructura fija** (su JSX), recibe **entradas predecibles** (sus props) y se puede **reutilizar** en cualquier parte de tu app.

Anatomía de un componente en TypeScript

Un componente es simplemente una **función de TypeScript que retorna JSX**. Con TypeScript, también definimos el tipo de las props que acepta:

TarjetaProducto.tsx — componente completo y anotado

```
// ① Importaciones necesarias
import React from "react";

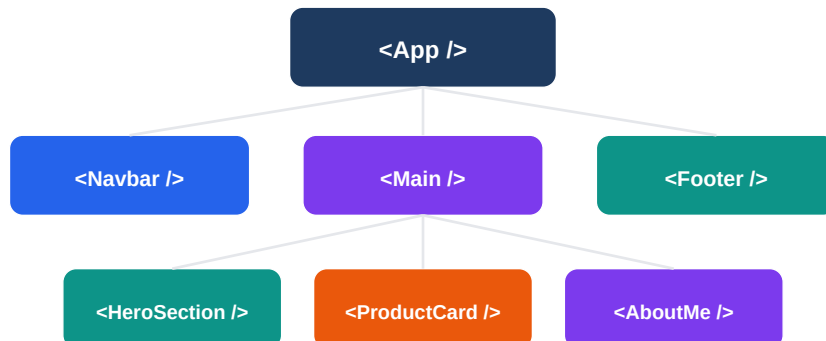
// ② Interface que define las props (entradas del componente)
interface TarjetaProductoProps {
  nombre: string;
  precio: number;
  imagen: string;
  disponible?: boolean; // opcional → tiene valor por defecto
}

// ③ El componente: función que recibe props y retorna JSX
export function TarjetaProducto({
  nombre,
  precio,
  imagen,
  disponible = true, // valor por defecto para props opcionales
}: TarjetaProductoProps) {
  // ④ El return: el JSX que se renderiza
  return (
    <div className="card">
      <img src={imagen} alt={nombre} />
      <h3>{nombre}</h3>
      <p>${precio}</p>
      {!disponible && <span>Agotado</span>}
    </div>
  );
}
```

```
}
```

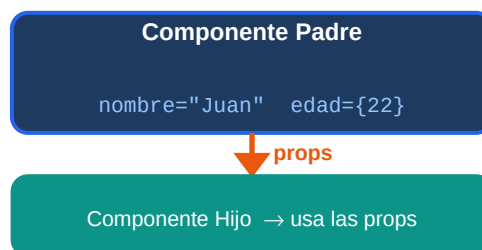
El árbol de componentes

Una aplicación React es un **árbol de componentes** anidados. Los componentes grandes se componen de componentes más pequeños. Esto se llama *composición*:



¿Cómo se pasan datos entre componentes? Props

Las **props** (properties) son la forma en que un componente padre le pasa información a un componente hijo. El flujo siempre es **de padre a hijo, nunca al revés**.



Usando el componente con diferentes props

```
// Padre: tiene los datos y los pasa
function App() {
  return (
    <div>
      /* Cada vez que usas el componente, le pasas props distintas */
      <TarjetaProducto
        nombre="Camiseta Vintage"
        precio={45000}
        imagen="/img/camiseta.jpg"
      />
      <TarjetaProducto
        nombre="Jeans Classic"
        precio={89000}
        imagen="/img/jeans.jpg"
        disponible={false}
      />
    </div>
  );
}
```



```
</div>  
);  
}
```

Las grandes ventajas de usar componentes

■ ■ Reutilización	Escribes el componente UNA vez, úsalo N veces. Una TarjetaProducto puede mostrarse 100 veces con datos distintos sin duplicar código.
■ Aislamiento	Cada componente es responsable solo de su propio renderizado. Si hay un bug en TarjetaProducto, sabes exactamente dónde buscar.
■ Mantenibilidad	Cambiar el diseño de todas las tarjetas = editar UN solo archivo. Sin componentes, tendrías que editar el HTML en 50 lugares distintos.
■ Tipo-seguridad con TypeScript	Si defines las props con una interface, TypeScript te avisa si intentas usar el componente sin pasar una prop obligatoria, o si le pasas el tipo equivocado.
■ Composición	Componentes dentro de componentes. Una Page se compone de Sections, que se componen de Cards, que se componen de Buttons. Código legible y escalable.

0 Ejercicio Práctico: Tu Proyecto, Tus Componentes

■ 30 min

4 30 minutos para aplicar todo lo aprendido en tu propio proyecto

Es momento de trabajar con tu proyecto real. Dependiendo de qué estás construyendo, los componentes que vas a crear son distintos. Esta sección tiene **ejercicios guiados** para los dos tipos de proyectos más comunes de la clase.

■ **Paso previo (todos):** Abre tu proyecto, navega a `src/` y crea una carpeta llamada `components/`. Todos tus componentes van aquí. Un archivo por componente.

■ Para quienes hacen un portafolio personal

Ejercicio 1: Componente Navbar

Crea `src/components/Navbar.tsx`:

Navbar.tsx

```
// src/components/Navbar.tsx

interface NavbarProps {
  nombre: string; // Tu nombre para mostrar en el logo
  links: string[]; // Ej: ["Inicio", "Proyectos", "Contacto"]
}

export function Navbar({ nombre, links }: NavbarProps) {
  return (
    <nav className="navbar">
      <span className="logo">{nombre}</span>
      <ul>
        {links.map((link) => (
          <li key={link}>
            <a href={`#${link.toLowerCase()}`}>{link}</a>
          </li>
        ))}
      </ul>
    </nav>
  );
}
```

Ejercicio 2: Componente ProyectoCard

Crea `src/components/ProyectoCard.tsx`:

ProyectoCard.tsx

```
// src/components/ProyectoCard.tsx

interface ProyectoCardProps {
  titulo: string;
```

```

descripcion: string;
tecnologias: string[];
urlDemo?: string; // opcional
}

export function ProyectoCard({ titulo, descripcion, tecnologias, urlDemo }: ProyectoCardProps) {
  return (
    <div className="proyecto-card">
      <h3>{titulo}</h3>
      <p>{descripcion}</p>
      <div className="tags">
        {tecnologias.map((tech) => (
          <span key={tech} className="tag">{tech}</span>
        ))}
      </div>
      {urlDemo && <a href={urlDemo} target="_blank">Ver demo</a>}
    </div>
  );
}

```

■ Para quienes hacen un sitio de ventas

Ejercicio 1: Componente ProductoCard

Crea src/components/ProductoCard.tsx:

```

ProductoCard.tsx

// src/components/ProductoCard.tsx

interface ProductoCardProps {
  nombre: string;
  precio: number;
  imagen: string;
  categoria: string;
  disponible?: boolean;
}

export function ProductoCard({
  nombre, precio, imagen, categoria, disponible = true
}: ProductoCardProps) {
  const precioFormateado = precio.toLocaleString("es-CO", {
    style: "currency", currency: "COP", maximumFractionDigits: 0
  });

  return (
    <div className={`product-card ${!disponible ? "sold-out" : ""}`>
      <img src={imagen} alt={nombre} />
      <span className="category">{categoria}</span>
      <h3>{nombre}</h3>
      <p className="price">{precioFormateado}</p>
    </div>
  );
}

```

```
<button disabled={!disponible}>
{disponible ? "Agregar al carrito" : "Agotado"}
</button>
</div>
);
}
```

Ejercicio 2: Componente Header con carrito

Crea src/components/Header.tsx:

```
Header.tsx

interface HeaderProps {
  nombreTienda: string;
  itemsEnCarrito: number;
}

export function Header({ nombreTienda, itemsEnCarrito }: HeaderProps) {
  return (
    <header className="header">
      <h1>{nombreTienda}</h1>
      <div className="cart">
        <span>■</span>
        {itemsEnCarrito > 0 && <span className="badge">{itemsEnCarrito}</span>}
      </div>
    </header>
  );
}
```

Conectando todo en App.tsx

Una vez creados tus componentes, el `App.tsx` es donde los **importas y combinas**. Aquí empieza a verse la magia de la composición:

App.tsx — composición de componentes

```
// src/App.tsx - ejemplo para portafolio
import { Navbar } from "../components/Navbar";
import { ProyectoCard } from "../components/ProyectoCard";

// Datos de ejemplo - luego vendrán de una API
const proyectos = [
  { titulo: "App de Clima", descripcion: "Clima en tiempo real",
    tecnologias: ["React", "TypeScript", "API"] },
  { titulo: "E-commerce", descripcion: "Tienda online completa",
    tecnologias: ["React", "Node.js"], urlDemo: "https://demo.com" },
];

function App() {
  return (
    <div>
      <Navbar nombre="Juan Dev" links={["Inicio", "Proyectos", "Contacto"]} />
      <main>
        {proyectos.map((p) => (
          <ProyectoCard key={p.titulo} {...p} />
        ))}
      </main>
    </div>
  );
}

export default App;
```

Errores comunes y cómo resolverlos

Tipo	Mensaje de error	Estado	Cómo arreglarlo
■ Error	Property 'X' does not exist on type 'Y'	■ Solución	Falta agregar esa prop a la interface del componente.
■ Error	Type 'string' is not assignable to type 'number'	■ Solución	Estás pasando texto donde se espera número. Usa {45000} con llaves, no "45000" con comillas.
■ Error	Missing required prop: 'nombre'	■ Solución	Olvidaste pasar una prop obligatoria. Revisa la interface y agrega la prop faltante.

■ Error	Cannot find module './components/Navbar'	■ Solución	El archivo se llama Navbar.tsx pero estás importando 'Navbar'. Revisa mayúsculas y la extensión.
■ Error	Each child in a list should have a unique 'key' prop	■ Solución	Cuando usas .map() para renderizar una lista, agrega key={algo_único} al elemento raíz.



Reto adicional (si terminas antes)

Crea un componente Badge genérico que reciba un texto y un color, y úsalo tanto en las tarjetas de proyectos (para las tecnologías) como en las tarjetas de productos (para la categoría). Eso es **reutilización real**: un componente, dos casos de uso.

0 Cierre y Reflexión

5 Qué aprendimos hoy y hacia dónde vamos

■ 5 min

Resumen de lo visto hoy

- **TypeScript** Es JavaScript con tipos. Detecta errores antes de ejecutar. No corre en el navegador: se compila a JS.
- **tsconfig.json** Controla cómo TypeScript compila tu código: qué versión de JS generar, qué tan estricto ser, y cómo resolver módulos.
- **Flujo Vite** Tu .tsx → TypeScript lo tipifica → esbuild lo transpila a .js → Vite lo sirve al navegador. Todo en milisegundos.
- **Componentes** Funciones que retornan JSX. Reciben props tipadas con interfaces. La base de cualquier app React: reutilizables, aislados, composables.
- **Props + Interfaces** Las interfaces definen el contrato de cada componente. TypeScript te avisa si usas mal un componente. Menos bugs, más confianza.

¿Qué viene en la próxima clase?

- useState y useEffect: cómo un componente recuerda información y reacciona a cambios
- Renderizado condicional avanzado y listas dinámicas desde datos reales
- Estilos en React: CSS Modules, clases condicionales y diseño responsive
- Formularios con TypeScript: eventos, validación y tipos del DOM

Preguntas para pensar antes de la próxima clase

1. ¿Por qué es mejor tener muchos componentes pequeños que un App.tsx gigante?
2. ¿Qué pasaría si dos componentes distintos necesitan los mismos datos? ¿Cómo los compartirías?
3. ¿Qué ventaja tiene TypeScript en un proyecto en equipo vs. uno personal?
4. En tu proyecto, ¿qué partes de la UI se repiten o se podrían convertir en un componente?